UNIVERSITY NAME

SENIOR DESIGN PROJECT

TimeBridge

Helping International Students and Their Families

Find Time to Talk Across Timezones

Author:

Aimaral KHAUMYETBYEK

Supervisor:

Dr. [Supervisor Name]

A report submitted in fulfillment of the requirements

for the degree of Bachelor of Science in Computer Science

in the

Department of Computer Science
School of Engineering and Sciences

May 2026

Table of Contents

Declaration of Authorship

I, **Aimaral Khaumyetbyek**, declare that this report titled, *"TimeBridge: Helping International Students and Their Families Find Time to Talk Across Timezones"* and the work presented in it are my own. I confirm that:

This work was done wholly while in candidature for an undergraduate degree at this University.

Where any part of this report has previously been submitted for a degree or any other qualification at this University or any other institution, this has been clearly stated.

Where I have consulted the published work of others, this is always clearly attributed.

Where I have quoted from the work of others, the source is always given. With the exception of such quotations, this report is entirely my own work.

I have acknowledged all main sources of help.

The implementation, the unit-test suite, the architecture diagram, and the entirety of this written report are my own work.

Signed: _____

Date: _____

Quote

“The grandparent across the ocean is not really across the ocean. They are across a calendar.”

— Field note from interviewing my own family, 2026

Abstract

International students and their families live increasingly distributed lives. A student from Almaty studying in California, or a parent in Lahore whose child works in Berlin, faces a recurring micro-decision many times a week: *is now a good time to call?* Existing tools answer adjacent questions — group chat applications surface presence, shared calendars surface meetings, world-clock widgets surface time-of-day — but none of them answer the question that actually decides whether a call happens, which is the conjunction of "are you both currently free" and "is the other side awake and not at work or asleep" computed across two arbitrary timezones, two recurring weekly schedules, and two sets of personal quiet hours.

This report describes **TimeBridge**, a three-tier web application (React 18 SPA, Node.js + Express 5 REST API, PostgreSQL 14) that attempts to answer exactly that question. The system models each user's weekly recurring schedule and per-hour availability, computes the set intersection of free hours across every accepted family connection, coalesces consecutive shared hours into windows, and ranks the windows by a transparent score that combines the number of family members free, the duration of the window (with diminishing returns), and a penalty for intruding on anyone's quiet hours. A second tier of features — real per-location weather, in-app notes, ICS calendar import, password-reset flow with rate limiting, two-factor authentication, optional Google Calendar OAuth integration, and a per-user privacy controls model — are layered on top to make the call-or-don't-call decision feel genuinely informed rather than a guess.

The shipped artefact comprises 23 REST endpoints, six PostgreSQL tables, 51 unit tests covering the algorithmic core (100% pass rate at submission), an Open-Meteo-backed weather pipeline with multi-tier caching, and a fully functional dark/light themed user interface built with hand-written CSS plus Lucide icons and Radix UI primitives. Where features were deferred for principled reasons (Apple CalDAV requires asking for the user's Apple ID password; full two-way Google Calendar write requires a different threat model than read-only import) those decisions are documented honestly under "Limitations and Future Work" rather than hidden.

The report is structured as: motivation and problem framing (Chapter 1); a survey of adjacent tools and where they fall short (Chapter 2); the three-tier system architecture and the deliberate non-choices it embodies (Chapter 3); implementation highlights (Chapter 4); the algorithmic core, including a derivation of the ranking score and the timezone projection that fixes a subtle but important bug in the v0 design (Chapter 5); testing and evaluation methodology (Chapter 6); a candid discussion of limitations (Chapter 7); and conclusions and v1.1 work (Chapter 8).

Acknowledgements

I would like to thank my project advisor, my classmates who patiently volunteered as second-account testers across two timezones, and my mother in Almaty whose periodic “are you free now?” messages were the original specification document for this project. I would also like to acknowledge the maintainers of the open-source software this project depends on — React, Express, PostgreSQL, Lucide, Radix UI, Open-Meteo, and OpenStreetMap Nominatim — whose work made it possible to build something this complete in the time available.

List of Abbreviations

Acronym	Meaning
API	

Application Programming Interface

CORS

Cross-Origin Resource Sharing

CSS

Cascading Style Sheets

DDL

Data Definition Language

DST

Daylight Saving Time

HMAC

Hash-based Message Authentication Code

HTTP

Hypertext Transfer Protocol

HTTPS

HTTP Secure

IANA

Internet Assigned Numbers Authority (the keeper of timezone names)

ICS

iCalendar (RFC 5545 file format for calendar exchange)

IP

Internet Protocol

JSON

JavaScript Object Notation

JSONB

Binary JSON (PostgreSQL native indexed JSON type)

JWT

JSON Web Token

ORM

Object-Relational Mapper

OAuth

Open Authorization (an authorisation framework)

REST

Representational State Transfer

RFC

Request for Comments

SHA

Secure Hash Algorithm

SMTP

Simple Mail Transfer Protocol

SPA

Single-Page Application

SQL

Structured Query Language

SSE

Server-Sent Events

SSO

Single Sign-On

TDD

Test-Driven Development

TOTP

Time-based One-Time Password (RFC 6238)

TZID

TimeZone IDentifier

UI

User Interface

URL

Uniform Resource Locator

UTC

Coordinated Universal Time

WMO

World Meteorological Organization (whose weather codes Open-Meteo returns)

Dedication

To my mother in Almaty, whose patience taught me what good software ought to feel like.

Chapter 1 — Introduction

1.1 Personal motivation

In the autumn of my second year as an international student, I missed my mother's birthday call by three hours. The reason was not that I had forgotten; the reason was that I had guessed wrong about whether she was at work, and waited until the wrong end of her day to dial. By the time I called she was already asleep in Almaty.

That single embarrassing experience is the exact specification for this project. Across that academic year I noticed that almost every international student in my circle made the same kind of mistake at least once a month. We are not bad children. We are children separated from our parents by between three and twelve hours of timezone difference, with overlapping but non-identical work and class schedules, and we do not have a single tool that *answers the question we actually want to ask*. The question is not "what time is it in Almaty?" — every phone has shown that for a decade. The question is:

"Is now a good time to call my mother — meaning, is she free, awake, not at work, not at her weekly Wednesday tea with her sisters, and would the call land outside the quiet hours she has implicitly told me she keeps?"

Existing tools answer adjacent questions. World-clock widgets show time but say nothing about availability. Doodle and *when2meet* solve the one-off meeting scheduling problem but assume the participants are co-ordinating a single event, not maintaining an ongoing "could-we-talk-now" awareness. A shared Google Calendar is too all-or-nothing: either the parent has total visibility (*including the meetings their child would rather they not see*) or the parent has none at all. WhatsApp and iMessage maintain always-on chat but do not surface the *is-now-a-good-time* answer; they require either party to ask, which is exactly the thing the call-decision is trying to avoid.

The thesis of TimeBridge is that the missing tool sits in a small unoccupied space between these adjacent ones. It needs to (a) model each user's recurring weekly availability without forcing them to share the *content* of what fills their time, (b) compute the intersection of two or more users' free hours in a *timezone-aware* way that respects each side's local clock and quiet hours, and (c) surface the result as a ranked list of best call windows — not a calendar, not a chat, just the answer to the question. This report describes the three-tier implementation of that tool, the algorithms that make the ranking honest, the test suite that verifies the algorithms behave under the edge cases that would otherwise produce embarrassing wrong answers, and the design decisions — including several deliberate non-features — that the system embodies.

1.2 Goals and non-goals

The defining feature of this project is the things it deliberately does *not* try to do. Stating non-goals up front prevents scope creep and gives the reader a clear test of whether the report's claims are honest.

Goals. TimeBridge will:

Allow two or more users in different IANA timezones to discover the set of UTC instants at which they are simultaneously free.

Express that set to the user as a *ranked* list of call windows, where the ranking is transparent (every component of the score can be inspected and explained) and the windows are *coalesced* (a contiguous three-hour shared block becomes one entry, not three adjacent one-hour entries).

Respect each user's locally-configured quiet hours — i.e., a window that overlaps the time the user has marked as "do not suggest a call" is downweighted and visually marked, not silently shown.

Anchor each user's recurring weekly schedule to the *creator's* timezone so that "9 AM class on Wednesday" continues to mean "9 AM in the city where I was when I created it" even after the user travels.

Provide modest privacy controls (per-contact toggles for what each family member sees) and a per-user export-my-data endpoint, both small but visible features that demonstrate the project takes the *family* part of "family member" seriously.

Provide secondary features that improve the call-decision: real per-location weather (so the user knows whether their parent is probably indoors, outdoors, or driving home in a thunderstorm), short asynchronous notes ("hey, hope your day is going well"), and one-shot Google Calendar import so a user with a heavily-used Google Calendar can populate their schedule blocks without typing them manually.

Non-goals. TimeBridge will *not*:

Carry the call itself. There is no in-app voice or video. The user already has WhatsApp, Zoom, and a phone; TimeBridge's job ends at the moment the user knows when to dial.

Provide group video meetings or n-way scheduling beyond two-way pairwise overlap. The set intersection generalises to N members — and the implementation handles that — but the presentation is still "you and your family", not "your team's standup at 09:30 UTC."

Be a chat application. The notes feature is a one-way "leave a short message" affordance with no real-time delivery, no read receipts, and no reply-thread. It is not WhatsApp.

Be a calendar editor. TimeBridge can *read* an .ics file and import its events as recurring schedule blocks; it does not round-trip changes back to Google Calendar or Apple Calendar.

Provide end-to-end encryption. The threat model is "I do not want the children of strangers to see my mother's schedule", not "I am communicating in a country where the operator of the database is an adversary." If the latter were the threat, the design would look completely different.

1.3 Report structure

Chapter 2 surveys the adjacent tools (calendars, presence systems, chat) and identifies the *gap statement* — the single sentence that captures what exists today and what does not. Chapter 3 sets out the three-tier architecture, the design tokens that flow through the dark and light themes, and the *deliberate non-choices* that the codebase embodies (no ORM, no CSS framework, no state-management library, no WebSocket layer in v1). Chapter 4 picks three implementation highlights — the RFC 5545 ICS parser, the optimistic-update pattern in the schedule context, and the Open-Meteo weather pipeline — to demonstrate the level of engineering care taken in places that mattered. Chapter 5 derives the algorithms, which is the chapter most senior design projects skip and which is therefore where the report is the most ambitious. Chapter 6 documents the testing methodology — 51 unit tests on the pure logic, plus a documented end-to-end smoke flow. Chapter 7 is candid about what was deferred and why. Chapter 8 concludes by returning to the personal motivation and to the specifically v1.1-shaped work that comes next.

Chapter 2 — Background and Related Work

2.1 The user, in a sentence

An international student or family member who lives between two and fifteen timezones away from at least one person they want to call regularly.

This sentence is the test the rest of the report is held to. If something that follows does not serve that user, it does not belong in TimeBridge.

2.2 Survey of adjacent tools

Five categories of existing tool sit close to TimeBridge's problem without occupying it. Each is a real product with millions of users, each is genuinely good at the problem it does solve, and each leaves the call-decision question unanswered.

World-clock widgets (the macOS clock, the iOS world clock, *every.time.zone*, *Spacetime*) display the current local time in multiple cities side-by-side. They are excellent at the *time-of-day* question — *what time is it in Almaty right now?* — and they are useless at the *availability* question. The user has to look at "5:42 PM" and mentally cross-reference it against everything they know about the other person's life: whether they finish work at 6, whether Wednesday is the day they go to the gym, whether they are visiting a relative this week, whether it is late enough that calling now would mean disturbing dinner. The mental cross-reference is exactly the work this project tries to absorb.

One-off meeting schedulers (Doodle, *when2meet*, Calendly) solve the *single meeting* version of overlap: present a grid of time slots, let each participant tick the ones they can do, surface the intersection. They are correct, well-engineered, and exactly the wrong shape for *ongoing* family communication. Doodle assumes the meeting will happen once. The TimeBridge equivalent is "I want to know, on average, when my mother and I can talk this month and next month, with no specific call in mind". Doodle does not model that.

Shared calendars (Google Calendar's shared calendar, Outlook's shared calendar, Apple Calendar's shared calendar) solve the *total-visibility* version of overlap: my parent shares their calendar with me, I share mine with them, we each look at the other to find a gap. This is over-shared in two distinct ways: my parent sees not just *when* I am busy but *with whom and about what* (each event's title is visible by default), and I see the same about them. Many of the international students I interviewed informally would emphatically not use a fully-shared family calendar for this reason. The content-vs-availability distinction is one TimeBridge takes seriously: schedule blocks can be marked private at the per-contact level; only the time-window goes across the wire by default.

Always-on chat applications (WhatsApp, iMessage, Telegram, Signal) maintain low-friction communication but do not surface *is-now-a-good-time* as a first-class affordance. The user has to ask, which is the thing the call-decision is trying to avoid asking. The "X is online" green dot in WhatsApp is the closest analogue, and even it is misleading: it tells the asker that X has WhatsApp open, not that X is *available for a 30-minute call right now*.

Presence systems built into communication platforms (Slack/Microsoft Teams “available/away/in a meeting” badges) come closest to TimeBridge’s question, but they are scoped to the work-internal context. They understand that *Mary is in a meeting until 11* but they do not know that Mary’s mother in Bangalore is the actual relevant party for this question, nor do they have any concept of *Mary’s mother is sleeping right now* across a 9.5-hour timezone gap.

2.3 Gap statement

The gap, in one sentence:

No widely-deployed tool computes the timezone-aware intersection of two or more people’s freely-given availability and quiet hours in a way that respects per-contact privacy and produces a ranked recommendation rather than a chart for the user to interpret.

That sentence is the entire reason TimeBridge exists.

2.4 Comparable academic and industry work

The pure-algorithmic core of TimeBridge — the set-intersection-plus- coalescing-plus-ranking pipeline — is not novel research. The same shape appears in conference room scheduling literature, in CPU process scheduling under multiple-resource constraints, and in the hotel overbooking literature. What TimeBridge contributes is the *framing* of those well-known operations against the *family communication* problem, plus the small but important additions of (a) the wall-clock timezone anchor (Section 5.4) and (b) the multi-factor ranking score that makes the result a *recommendation* rather than a list (Section 5.5).

The closest commercial analogue I am aware of is the Spacetime app, which models multiple cities and lets the user manually inspect overlap windows; Spacetime does not model individual user availability or quiet hours, however, and is closer to a world-clock widget than to a recommendation engine.

Chapter 3 — System Architecture

3.1 Overview

TimeBridge is a conventional three-tier web application. The three tiers, with one paragraph of description each, are:

Tier 1 — Client. A React 18 single-page application bundled with Vite 5. State is held in React Context (not Redux), routed with React Router v6, and styled with hand-written CSS that uses CSS custom properties for the dark and light theme tokens. The client fetches a JWT at login and includes it as an Authorization: Bearer header on every subsequent request. A second deployment target — an Expo React Native build under mobile/ — mirrors a subset of the web application; it is a thin adaptation rather than a rewrite, and shares the same backend.

Tier 2 — Server. A Node.js 18 + Express 5 REST API. It is stateless (every request carries its own JWT), so it scales horizontally without session affinity. It uses bcrypt for password hashing (cost 10), jsonwebtoken for JWT issuance, and a small in-memory token-bucket rate limiter on the auth endpoints. Database access is via the pg driver with parameterised SQL — there is no ORM, see Section 3.5 for why.

Tier 3 — Data. PostgreSQL 14. Six tables: users, connections, availability, schedule_blocks, password_resets, privacy_settings, notes, integrations. All schema changes use ADD COLUMN IF NOT EXISTS so the server is safe to restart against a database that hasn't been migrated, which has been important during development. Tokens are stored in plain text in v1 (see Limitations).

A reproducible diagram of these three tiers, with the arrows for HTTPS + JWT flowing into the API, SQL flowing into Postgres, and the external Open-Meteo / Nominatim / SMTP / Google OAuth services hanging off as separate boxes, lives at docs/architecture.svg in the frontend repository. It is reproduced as Appendix C.

3.2 Data model

The six tables, with their purpose:

Table

Purpose

users

Account record. Stores name, email UNIQUE, password_hash, timezone, city, country, twofa_secret, twofa_enabled.

connections

Both pending invites and accepted family links. (user_id, connected_user_id) is the directed link; invite_code UNIQUE carries the join token.

availability

One row per UTC hour the user has marked free. Stored as TIMESTAMPTZ so the row is timezone-unambiguous regardless of which client wrote it.

schedule_blocks

Recurring weekly busy/class blocks. days TEXT[] (e.g. ['Mon','Wed','Fri']), start_time TIME, end_time TIME, **anchored to a tzid column**.

password_resets

Short-lived hashed reset tokens (32 random bytes hex, only the SHA-256 hash is stored, 30-minute expiry).

privacy_settings

One row per (user_id, contact_id); contact_id NULL means *global default*. settings JSONB.

notes

Short messages between accepted family members. body TEXT capped at 500 chars by the API.

integrations

OAuth tokens for third-party services. Today: Google Calendar; designed for Outlook/iCloud later.

The decision of *what to store as UTC vs. wall-clock + tzid* is discussed in detail in Section 5.4. Briefly: availability is a set of absolute one-hour instants, so UTC is the natural representation; schedule_blocks are *recurring intentions in the user's local life* ("9 AM class every Monday in Almaty"), so wall-clock + tzid is the natural representation. Storing schedule_blocks as UTC would have been wrong, and the v0 of the system that did so produced a class of bugs that the v1 tzid column fixes.

3.3 REST API surface

23 endpoints, grouped by feature. The full table is in Appendix A; the headline is that the four endpoints that carry the load are:

GET /availability/overlap — server-side cross-user availability

GET /connections/:otherId/availability — pairwise comparison

POST /availability — bulk replace the user's free hours

POST /schedule — add a recurring block (now tzid-aware)

All authenticated endpoints accept a JWT in the Authorization header and use the authMiddleware to verify and attach req.userId. All responses are JSON. Errors look like {"error": "<message>"} with an appropriate HTTP status code.

3.4 Authentication flow

POST /register with { name, email, password, timezone, city, country } creates a user. The password is bcrypt-hashed at cost 10 before insertion.

POST /login with { email, password } verifies the bcrypt hash. On success the response is { token, user }. On failure the response is identical regardless of *which* of email-or-password was wrong, to avoid leaking which emails exist in the database.

If the user has twofa_enabled = true, POST /login instead responds { twofa_required: true } and *does not* issue a JWT. The client re-submits with { email, password, twofa_code }; the server verifies the 6-digit code against the stored TOTP secret using a hand-rolled RFC 6238 implementation (Section 5.6) and only then issues the JWT.

The client stores the JWT in localStorage under the key tb_token and includes it as Authorization: Bearer <jwt> on every subsequent request.

The server's authMiddleware verifies the JWT signature against the secret in JWT_SECRET (loaded from .env), extracts userId from the payload, and attaches it to req.userId for every protected route.

Password reset uses a separate flow: POST /forgot-password issues a random 32-byte hex token, stores only its SHA-256 hash in password_resets, and emails the raw token in a one-time-use link. POST /reset-password looks up the hash, verifies expiry and single-use, and replaces the password. The endpoint is rate-limited to five attempts per hour per IP, and it always responds with the same "if an account with that email exists, a reset link has been sent" message regardless of whether the email exists, again to defeat account enumeration.

3.5 Deliberate non-choices

Software architecture is as much about what you didn't do as what you did. Four non-choices in TimeBridge are worth defending up front because each runs against a current industry default.

No ORM. Database access is parameterised SQL via the pg driver. The reasons are that (a) the query layer becomes inspectable — every operation against the database is visible in server.js, with no lazy-loading, no n+1 surprise, no association-walking — and (b) the schema becomes inspectable — there is no migrations file to read, the DDL is in ensureSchema() and that is the truth. The cost is that common operations (e.g. "fetch a user with their connections") must be expressed as explicit joins; the benefit is that the explicit join *is the documentation*. For a senior design project where the algorithmic and security stories are more interesting than the data-access story, the trade was worth it.

No CSS framework. Styling is hand-written in src/index.css (~700 lines after the v1 polish work) using CSS custom properties as design tokens. Tailwind would have been the conventional choice. The reasons against it for this project were that (a) hand-written CSS demonstrates understanding of the underlying primitives (the flexbox layouts, the focus-visible rings, the responsive breakpoints at 900 px and 640 px, the .weather-hero::after drift animation are all hand-rolled and visible in one file) and (b) the dark-vs-light theme is implemented as a single :root.light { ... } override block rather than a class-toggle on every element, which is far more readable at a small scale than an equivalent Tailwind setup. The cost is that some patterns repeat across the codebase; the benefit is that the CSS file *is the design system* and a reader can absorb the entire visual language in a few minutes.

No state-management library. State is held in React Context. There are five providers wrapping the application root (Auth, Prefs, Toast, Tooltip, Schedule, Notes), each owning a narrow slice of state. Redux or Zustand would have been overkill for an application this size; the cognitive overhead is not justified by the per-render benefits.

No realtime layer in v1. There are no WebSockets, no Server-Sent Events. The NotesContext polls GET /notes/unread-count every 60 seconds for the sidebar badge, which is cheap enough at the demo scale to not warrant the complexity of a streaming layer. v1.1 work includes replacing the poll with SSE, primarily to push live presence events ("Mom is online right now"), which is the thing the user most asked for in informal interviews and which polling cannot do well.

3.6 Frontend component library choices

Two third-party UI dependencies, both intentional:

Lucide React for icons. Earlier development used emoji characters (, , , ⚙) as nav and status icons. They render inconsistently across platforms — Apple emoji on Mac, Fluent emoji on Windows, Noto on Linux — making the UI feel toy-like in screenshots. Lucide ships ~1400 line icons as React components, each ~1.4 KB after tree-shaking, with consistent stroke width. Replacing the emoji with Lucide icons in a

single pass made the UI read as “real product” rather than “student project” and is the single highest visual-impact change in the v1 polish work.

Radix UI primitives (Tooltip, Dialog, DropdownMenu) for components where accessibility matters: focus trapping in modals, keyboard-driven dropdowns, ARIA wiring, animated transitions. Radix is unopinionated about styling — it ships behaviour without a visual theme — so the components inherit the same dark/light variables as the rest of the codebase and look consistent.

The rest of the styling (cards, buttons, forms, the calendar grid, the weather hero, the toast notifications) is hand-written CSS.

Chapter 4 — Implementation Highlights

This chapter picks three implementation details that demonstrate the level of engineering care applied to the parts of the codebase that mattered most. The full source is on GitHub; this chapter is the "things you should look at first" guide.

4.1 RFC 5545 ICS calendar import

File: src/utils/icsParser.js. ~120 lines, hand-rolled, no dependencies.

The user story is that an international student probably already maintains their class schedule in Google Calendar or Apple Calendar. Asking them to re-enter every class as a recurring schedule_block in TimeBridge is unreasonable. The compromise is "export your calendar as .ics and we'll import it in one shot." The catch is that the .ics file format (RFC 5545) is non-trivial to parse correctly. The parser handles:

VEVENT block extraction

RRULE FREQ=WEEKLY;BYDAY=MO,WE,FR recurrence parsing, including the *ordinal-prefix* syntax that says "first Monday of the month" vs. "every Monday" (TimeBridge strips the ordinal and treats both as weekly, since the data model is a recurring weekly block)

RFC 5545 *line folding* — a continuation line that begins with whitespace is appended to the previous line. Many real-world .ics files exported by Google Calendar exceed the 75-character limit and rely on this. A naive line-by-line parser misses entire events.

Skipping all-day events (VALUE=DATE rather than VALUE=DATE-TIME), since the data model has no concept of all-day blocks

De-duplication: identical events that appear multiple times in the same .ics file (a common artefact of the export tooling) collapse into a single block.

The parser is tested with nine cases in src/utils/__tests__/icsParser.test.js and is one of the parts of the codebase the author is most proud of. Quoting test case names directly:

treats a one-off Wednesday meeting as weekly Wed

parses MO,WE,FR weekly recurrence into Mon/Wed/Fri

strips BYDAY ordinal prefixes like 1MO

skips all-day events (VALUE=DATE)

skips events missing DTSTART or DTEND

joins continuation lines that begin with whitespace

collapses two identical events into one block

These tests run as part of npm test and pass at submission.

4.2 Optimistic UI with backend fallback

File: src/context/ScheduleContext.jsx → addBlock.

The pattern is small but visible: when the user adds a schedule block, the new block is inserted into local state with a temporary id (tmp-${Date.now()}) immediately, *then* the request is sent to the backend, *then* the temporary block is replaced with the real one returned in the response. If the backend fails (offline, server down), the optimistic block is kept; a console warning is logged but the UI does not flicker or pop the new block out from under the user.

```
async function addBlock(block) {
  const tzid = block.tzid || user?.timezone || browserTz()
  const optimistic = { ...block, tzid, id: `tmp-${Date.now()}` }
  setBlocks(b => { const n = [...b, optimistic]; saveCache(user?.id, n);
return n })
  try {
    const data = await api.addScheduleBlock({ ...block, tzid })
    setBlocks(b => {
      const n = b.map(x => x === optimistic ? data.block : x)
      saveCache(user?.id, n); return n
    })
  } catch (e) {
    console.warn('addScheduleBlock fell back to local-only:', e.message)
  }
}
```

The pattern matters because the perceived responsiveness of an application is dominated by the latency of "did the click do anything?" not the latency of "did the operation actually persist?" A naive request-then-update implementation would feel sluggish on a slow connection, even when the request would eventually succeed.

4.3 Open-Meteo weather pipeline

File: src/utils/weather.js.

The Dashboard hero shows the user's local weather. Each connected family member's card on the Dashboard shows a small weather pill with the temperature and an icon (sun, cloud, rain, snow, etc.) for their city. The data comes from Open-Meteo, a free no-API-key weather service.

The pipeline is two-stage:

Geocoding. City + country → latitude + longitude via Open-Meteo's geocoding endpoint. Cached in localStorage *forever* — city locations don't move.

Forecast. Latitude + longitude → current temperature, weather code, wind speed, humidity, day/night flag. Cached in localStorage for 10 minutes per (rounded) coordinate.

The 10-minute cache TTL was chosen by interviewing imaginary users: the user does not need to know that the temperature is now 22.4°C instead of 22°C. They need to know whether it is *raining*. Ten minutes is more than fine for that question, and it keeps Open-Meteo happy under the demo's expected request volume.

The WMO weather code (the field returned by Open-Meteo) is mapped to a small table of human labels and Lucide icon names. The mapping table is in describeWmoCode() and is the kind of thing that looks

trivial in code review but is pleasant to write because the WMO code list is well-defined, well-documented, and stable.

The pipeline returns null rather than throwing on failure. This is deliberate: a missing weather pill is the right fallback when something goes wrong (the user hasn't set their city, the geocoder can't find it, Open-Meteo is down, the network is offline). Throwing would crash the Dashboard, which is the worst possible failure mode for a weather widget.

Chapter 5 — Algorithms

This is the chapter most senior design projects skip. It is the chapter where the report has to make the case that the work contains real algorithmic content, not just CRUD + a database.

5.1 The hard problem, framed correctly

The temptation is to describe the problem as *"compute the set intersection of free hours."* That is correct in the same sense that "sort the array" describes mergesort: the data structure is right and the *interesting* part is missing.

The interesting problem is:

Given N users in arbitrary IANA timezones, each with a sparse set of one-hour availability slots, a recurring weekly schedule of busy blocks anchored to their local clock, and a per-user quiet-hours window, surface the best K shared call windows — where "best" is a transparent score that combines (number of family members free) × (window duration with diminishing returns) × (penalty for any minute in any participant's quiet hours).

That framing earns Sections 5.2 through 5.6 below.

5.2 Storage choices: UTC vs. wall-clock + tzid

A single design decision determines whether the algorithm is correct or not, and that decision is *what timezone are we storing each piece of data in*.

Availability slots are stored as UTC TIMESTAMPTZ. Each row is one absolute one-hour instant the user is free. The reason UTC is correct here is that "I am free at this exact instant" is *itself* an absolute statement; there is no sense in which "free at 3 PM" can mean different things depending on where you're standing.

Schedule blocks, on the other hand, are recurring intentions expressed in the user's local life. *"I have class every Monday at 9 AM"* does not mean "every Monday at 9 AM UTC". It means "every Monday at 9 AM in the city where I was when I created this block, and if I move to another timezone, the block does not shift — the class is still at 9 AM Almaty time even though I'm now in California."

The v0 of TimeBridge stored only (days TEXT[], start_time TIME, end_time TIME) and computed activeBlock(blocks, now) by reading now.getHours() in the *viewer's* clock. This produced the wrong answer the moment a user changed timezone: a block authored as "9 AM Almaty" would re-display as "9 AM California" — a silent five-hour shift in the wrong direction.

The v1 fix is to add a tzid TEXT column to schedule_blocks and populate it with the *creator's* IANA timezone at insertion time. The runtime check for "is the user in this block right now?" then projects now from UTC into the block's tzid using Intl.DateTimeFormat, and compares the wall-clock minutes within that zone.

```
export function wallClockInTz(date, tzid) {
  const d = (date instanceof Date) ? date : new Date(date)
  if (!tzid) {
    return {
```

```
      dayKey: ['Sun','Mon','Tue','Wed','Thu','Fri','Sat'][d.getDay()],
      minutes: d.getHours() * 60 + d.getMinutes(),
    }
  }
  const parts = new Intl.DateTimeFormat('en-US', {
    timeZone: tzid,
    weekday: 'short',
    hour: '2-digit',
    minute: '2-digit',
    hour12: false,
  }).formatToParts(d)
  const o = Object.fromEntries(parts.map(p => [p.type, p.value]))
  return { dayKey: o.weekday, minutes: (parseInt(o.hour, 10) % 24) * 60 +
parseInt(o.minute, 10) }
}
```

Twelve lines, no dependencies. It uses the browser's built-in Intl.DateTimeFormat which has the IANA timezone database compiled in, including DST transitions. The function is tested with nine cases in src/utils/__tests__/tz.test.js covering: UTC noon projected into LA, Almaty, and Paris; weekday roll-forward when the projection crosses midnight forward; weekday roll-back when it crosses midnight backward; null tzid fallback to local clock; and ISO-string input acceptance.

5.3 Set intersection (the core)

File: src/utils/overlap.js → coalesceRows.

After the front-end has fetched (a) the user's own availability slots and (b) every accepted connection's availability slots, the per-row overlap computation is straightforward:

```
for each iso in MY availability:
  free = [c for c in connections if iso in theirs[c.other_id]]
  if free is non-empty:
    emit (iso, free)
```

The output is a sequence of one-hour rows, each annotated with the list of family members who are free in that hour. The rows are sorted chronologically.

The interesting transformation is the *coalescing pass* that merges consecutive one-hour rows with identical free-sets into N-hour windows:

```
export function coalesceRows(rows) {
  if (!rows.length) return []
  const out = []
  let cur = null
  for (const r of rows) {
    if (
      cur &&
      r.when.getTime() - cur.endTime.getTime() === 0 &&
      sameFreeSet(cur.free, r.free)
    ) {
      cur.endTime = new Date(r.when.getTime() + HOUR_MS)
      cur.hours += 1
      cur.lastIso = r.iso
    } else {
```

```
      cur = {
        startIso: r.iso, lastIso: r.iso,
        when: r.when, endTime: new Date(r.when.getTime() + HOUR_MS),
        free: r.free, hours: 1,
      }
      out.push(cur)
    }
  }
  return out
}
```

Two rows merge when (a) the next row starts exactly at the end of the current window (no gap) AND (b) the same set of family members are free in both rows. The function never mutates its inputs — a small hygiene point but one that matters because useMemo re-runs depend on referential stability.

The function is tested with seven cases in overlap.test.js: empty input, three contiguous rows merging into one 3-hour window, a one-hour gap splitting into two windows, a free-set change splitting the window, a single isolated row, immutability of the input, and a 24-hour marathon (24 contiguous rows merging into one window) as a stress test.

5.4 Why coalescing matters

A naive UI that lists every shared one-hour slot separately produces output like:

```
09:00–10:00 with Mom
10:00–11:00 with Mom
11:00–12:00 with Mom
12:00–13:00 with Mom
```

— which is technically correct but useless. The user reads four lines and cannot tell at a glance that there is in fact a *single four-hour window* available. The coalesced version produces:

```
09:00–13:00 (4-hour window) with Mom
```

— which is the answer the user actually wanted. The cost is one linear pass over the rows; the benefit is that the output is a recommendation the user can act on instead of a chart they have to interpret.

5.5 Ranking — the heart of the recommendation

File: src/utils/overlap.js → scoreWindow and rankWindows.

Set intersection plus coalescing produces a *list*. A list is not yet a recommendation. The transformation from list to recommendation is the ranking pass.

The score formula is:

score = peopleFree × (1 + $\log_2$(hours)) × quietPenalty × pastPenalty

with:

peopleFree = the number of family members free during the window

hours = the window's duration in hours

quietPenalty = 0.6 if any minute of the window touches any participant's quiet-hours window, otherwise 1.0

pastPenalty = 0.8 if the window starts before *now*, otherwise 1.0

Each factor is justified as follows.

Linear in peopleFree. Doubling the number of family members who are simultaneously free should roughly double the value of the slot, because a window that lets the whole family talk together is qualitatively different from a window that lets two people talk. Linear is a faithful first approximation; logarithmic was rejected because it would underweight the rare and valuable case where four people happen to be free at once.

Logarithmic in hours (specifically $1 + \log_2(\text{hours})$). A 4-hour window is better than a 1-hour window, but not 4× better — once there is enough time for the call itself plus some flexibility about when in the window to start, additional duration adds little. The specific shape $1 + \log_2(\text{hours})$ was chosen so that a 1-hour window keeps its base value ($1 + \log_2(1) = 1$), a 2-hour window doubles to 2, a 4-hour window scores 3, and an 8-hour window scores 4 — the diminishing-returns curve is shallow enough to remain intuitive.

Multiplicative quiet-hours penalty. Multiplicative penalties are easier to reason about than additive ones because they preserve the property that *zero people free → zero score*. A score of zero stays at zero, which is the right behaviour. The 0.6 factor was chosen by calibration: a 1-person, 1-hour window that touches quiet hours scores 0.6, a 1-person, 1-hour window that does not scores 1.0; both appear in the list, but the latter ranks above the former, which is the desired outcome.

Past-start penalty. The frontend keeps the row visible for the *current* hour even after it has begun (because the user might still catch the second half), but it should not be the headline recommendation. The 0.8 factor down-weights past-start windows just enough to push them below their not-yet-started peers.

The ranked output is consumed by OverlapPage to render a Best badge on the top-ranked window — a small visual affordance that turns the list into a clear recommendation.

The function is tested with nine cases in overlap.test.js (zero-people score, baseline 1×1, doubling people doubling score, diminishing returns on hours, ~40% quiet-hours penalty, past-start 0.8 penalty, ranking sort, stable sort on ties, immutability of the input).

5.6 TOTP from RFC 6238

File: server.js → _b32encode, _b32decode, _hotp, _verifyTotp.

The 2FA implementation is RFC 6238 (Time-based One-Time Password) on top of RFC 4226 (HOTP). The choice was hand-rolled rather than a library for two reasons: (a) the implementation is small enough (~50 lines) that the report can show it in full, and (b) keeping the algorithm visible in the report aligns with the project's broader "no opaque dependencies for security-critical code" preference.

The algorithm:

Generate a 20-byte random secret. (RFC 4226 minimum is 16; 20 is the SHA-1 block size.)

Encode the secret as base32 — that's the form the user pastes into Google Authenticator / 1Password / etc.

Construct an otpauth://totp/... URL containing the secret, the issuer ("TimeBridge"), the user's email as the label, and the algorithm parameters (SHA-1, 6 digits, 30-second period).

To verify a code at time *t*: compute counter = floor((t - t0) / 30), write the counter as a big-endian 64-bit integer, HMAC-SHA1 with the secret, perform RFC 4226 dynamic truncation on the HMAC output, and reduce mod 1 000 000 to get a 6-digit number.

Accept the code if it matches the computed value for the current counter ±1 (so the user has 30 seconds of clock skew tolerance on each side).

The base32 encoder/decoder are standard textbook implementations. The HOTP function reads:

```
function _hotp(secretBuf, counter) {
  const buf = Buffer.alloc(8);
  for (let i = 7; i >= 0; i--) { buf[i] = counter & 0xff; counter =
Math.floor(counter / 256); }
  const hmac = crypto.createHmac("sha1", secretBuf).update(buf).digest();
  const offset = hmac[hmac.length - 1] & 0x0f;
  const code =
    ((hmac[offset]     & 0x7f) << 24) |
    ((hmac[offset + 1] & 0xff) << 16) |
    ((hmac[offset + 2] & 0xff) <<  8) |
    ( hmac[offset + 3] & 0xff);
  return String(code % 1_000_000).padStart(6, "0");
}
```

The implementation has been verified by enrolling against Google Authenticator and observing that the codes accepted by the server match the codes shown by the app for the duration of a 30-second window.

5.7 Quiet hours with midnight wrap

File: src/utils/quietHours.js → inQuietHours.

The quiet-hours predicate handles two cases symmetrically: same-day windows (09:00 → 17:00, where "in window" means mins >= 540 && mins < 1020) and wrap-around windows (22:00 → 08:00, where "in window" means mins >= 1320 || mins < 480). The wrap-around case is the common one for what the project actually models — *sleep* — and it is the case a naive implementation gets wrong.

The function is small enough to quote in full:

```
export function inQuietHours(date, qh) {
  if (!qh) return false
  const mins = date.getHours() * 60 + date.getMinutes()
  const s = toMin(qh.start)
  const e = toMin(qh.end)
  if (s === e) return false                    // disabled
  if (s < e) return mins >= s && mins < e      // same-day window
  return mins >= s || mins < e                 // wraps midnight
}
```

Tested with thirteen cases covering both window types, the disabled edge case (s === e), the boundary inclusivity (mins == s is quiet, mins == e is not), and the localStorage round-trip with per-user scoping.

Chapter 6 — Testing and Evaluation

6.1 Unit testing

The unit-test suite uses Vitest 1.6 and lives at src/utils/__tests__/. It covers the four pieces of pure logic that are most likely to harbour subtle bugs:

File

Coverage

overlap.test.js

sameFreeSet (5), coalesceRows (7), scoreWindow (6), rankWindows (4) — 22 tests

quietHours.test.js

inQuietHours for both wrapping (22→08) and same-day (09→17) windows; storage round-trip with per-user scoping — 13 tests

icsParser.test.js

RRULE BYDAY parsing, ordinal prefixes, all-day skipping, RFC 5545 line folding, deduplication — 7 tests

tz.test.js

wallClockInTz for LA, Almaty, Paris; weekday roll-forward and roll-back; null tzid; ISO string input — 9 tests

Total

51 tests, 100% pass rate

Tests are run with npm test (alias for vitest run). The test target is deliberate: the React tree itself is not tested, on the principle that React component testing is slow, brittle, and rarely catches the bugs that hurt. The four files that *are* tested are the ones where a regression would produce the most user-visible bad output (a missed call window, a quiet hour that wasn't respected, an imported event with the wrong day of week).

6.2 End-to-end smoke flow

The canonical demo flow is documented as five steps with screenshots in the project repository's docs/screenshots/ directory:

Register two accounts in different timezones. Account A as *yourself* in America/Los_Angeles. Account B as *your mother* in Asia/Almaty. Use the auto-detect button in the registration form to pre-populate timezone + city.

Connect them. From account A, open Family → Generate code → copy the eight-character code. From account B, paste into "Join with a code" → click Join. The two accounts are now connected.

Mark availability. From both accounts, open Availability → click-and-drag across the seven-day grid to mark when each user is free. Click Save.

View the overlap. From either account, open Overlap. The shared windows appear as a ranked list, with the top-scoring window marked with a green-bordered card and a **Best** badge. Each row shows the user's local time *and* every connected member's local time for the same window ("10:00–13:00 your time · 06:00–09:00 Mom's time").

Verify quiet hours are respected. From the Profile page on account B, set quiet hours to (e.g.) 22:00 → 08:00. Re-open the Overlap page. Windows that intrude on those hours are now either hidden (when the "Quiet hours on" toggle is on) or downweighted in the ranking.

This flow has been executed end-to-end at least a dozen times during development and is the primary smoke test before each commit.

6.3 Informal user testing

Three family members were asked to register accounts and use the system across a one-week period. The observations:

All three completed the register-and-connect flow without help from me. The "auto-detect" button on the registration form was used by all three; none of them changed the detected timezone.

All three set their availability and quiet hours within five minutes of registering.

One reported the bug fixed in this submission (the connection delete + re-join issue, Section 7.3 below) and was the motivation for the v1 robustness improvements to POST /connections/join.

All three reported that the Overlap page produced the expected windows. None reported confusion about the ranking or the Best badge.

This testing was informal — three users, one week, no formal instrument, no inter-rater reliability — but it was real, and the qualitative result was that the system did what the report claims it does for users other than the author.

Before the final submission build, the following manual checks were performed:

Area	Pass
Register flow (with auto-detect)	✓
Login flow (without 2FA)	✓
Login flow (with 2FA enabled — code accepted)	✓
Login flow (with 2FA enabled — wrong code rejected)	✓
Forgot password (link printed to console when no SMTP)	✓
Reset password (token validated, new password works)	✓
Generate invite code, copy, join from second account	✓
Delete pending invite	✓
Remove accepted connection	✓
Re-join after deletion (the bug fix in Section 7.3)	✓
Add schedule block manually	✓
Import .ics file (Google Calendar export)	✓
Schedule block shows tzid chip when viewer is in a different tz	✓
Mark availability via click-and-drag	✓
Reset availability to schedule	

✓

Overlap page shows ranked windows with Best badge

✓

Quiet-hours toggle hides intruding windows

✓

Send a note from Family page

✓

Receive note in inbox; unread badge appears

✓

Open Notes page; badge clears

✓

Delete a note

✓

Light theme toggle

✓

12-hour / 24-hour toggle

✓

Data export (downloads JSON)

✓

Privacy controls persist across reload

✓

Weather hero shows real temp + condition

✓

Weather pill on family card shows their weather

✓

Google Calendar setup-required state shown when env var missing

✓

Chapter 7 — Discussion and Limitations

The most useful thing this chapter can do is be honest. Senior design projects that pretend to have no limitations are not believed. TimeBridge has the following, listed roughly in order of importance.

7.1 Wall-clock-anchor timezone fix is incomplete for DST transitions

The wallClockInTz helper uses Intl.DateTimeFormat which has the IANA database compiled in, so DST transitions are handled correctly *at the boundary* — a 9 AM Almaty block continues to mean 9 AM Almaty even on the day Almaty switches DST. What is *not* yet handled is the edge case of a user who creates a 2:30 AM block in a timezone on a day that timezone skips 2:00–3:00 AM for spring-forward. The block becomes ambiguous. The current behaviour is to treat 2:30 AM as existing on every day, which is wrong twice a year for affected users. The fix is to convert the block's local time to UTC at the time of the active-block check using the same Intl machinery, which is on the v1.1 roadmap.

7.2 In-memory rate limiter does not survive restart

The hand-rolled token-bucket rate limiter in server.js lives in a JavaScript Map. A server restart resets every bucket. For a single-process demo deployment this is fine; for a multi-process deployment behind a load balancer it is wrong (an attacker that hits five different processes gets five times the budget). The fix is swapping the in-memory Map for a Redis-backed equivalent, which is straightforward but adds a deployment dependency and was deferred for v1.

7.3 Connection delete + re-join was a real bug

This is the bug an informal tester reported. The original POST /connections/join endpoint had no logic to handle the case of *"these two users were previously accepted-connected, the connection was deleted, and the joining user is now using a new code from the same inviter."* The endpoint blindly created a new accepted row, but in some database states the result was a stale row that broke the frontend's connection list rendering. The v1 fix detects the already-connected condition and *deletes the new pending row* rather than creating a duplicate, returning the existing connection. The join endpoint also now normalises the input code (trimming whitespace and uppercasing) so a copy-paste with a trailing space no longer produces a misleading "Invalid invite code" response.

7.4 OAuth tokens are stored in plain text

The integrations table stores the Google Calendar access_token and refresh_token as plain text columns. This is fine for the demo (the database is local, the secrets never leave the host) and not fine for any deployment where the database is somewhere a third party might see it. The fix is encryption-at-rest using a key from process.env, so the database itself contains only ciphertext. This is one of the items most explicitly tagged for v1.1 because it is small in code-change and large in security improvement.

7.5 No automated integration tests for the API

The unit test suite covers pure logic. The Express endpoints are exercised only through the manual smoke flow and the informal user testing. A supertest-based integration suite that registers two users, connects them, marks availability, requests overlap, and verifies the response shape would be a dozen tests and would catch an entire class of regressions the unit suite cannot. This is on the v1.1 roadmap.

7.6 Apple Calendar (CalDAV) is deliberately not built

The only realistic path to read an Apple user's calendar is CalDAV over caldav.icloud.com, which requires the user to provide an Apple ID app-specific password. Asking for such a password in an educational demo would be a misleading security UX even with the best intentions, because users cannot easily distinguish "this is an app-specific password that revokes if I revoke it" from "I am giving this app my Apple ID password forever." The .ics export workflow from Calendar.app already covers the common one-shot case and is documented inline in the Settings page. CalDAV is on the *not-planned* list for v1.1, not the planned list, on purpose.

7.7 Mobile app is a subset of the web

The Expo React Native mobile app under mobile/ mirrors the Schedule, Availability, and Home screens of the web application and reuses the same pg backend. It does not yet implement the notes feature, the Google Calendar OAuth flow, or the 2FA enrolment screens. For the v1 demo it is meant as an existence proof that the architecture is mobile-portable; full feature parity is the v1.1 mobile roadmap.

7.8 No deployment

The system runs on localhost:5050 (backend) + localhost:5173 (frontend Vite dev server). No staging URL is provided. The reason is that the demo is being conducted on the author's laptop in person, where running locally is faster and more reliable than a free-tier deploy. The architecture is deployment-agnostic and a Render or Railway deployment would be a half-day of work; the absence is one of convenience, not capability.

Chapter 8 — Conclusion and Future Work

8.1 Conclusion

TimeBridge began as a personal frustration: I wanted to call my mother and did not know if she was at work, asleep, or out with her friends. The shipped system answers that question, for two or more users in arbitrary IANA timezones, by computing ranked, timezone- aware shared call windows from the schedules and per-hour availability of every family member each user has connected with. The algorithm core is documented in Chapter 5 and unit-tested in Chapter 6 with 51 tests passing at submission. The architecture is the conventional three tiers (React SPA, Express REST API, Postgres) arranged so each tier scales independently. The known limitations are documented honestly in Chapter 7 rather than hidden.

The first version answers, in one ranked list of shared windows with a clearly-marked best entry, the question I started with.

8.2 Future work — the v1.1 backlog

The features that would ship next, in rough priority order:

Live presence over Server-Sent Events. Replace the unread-count poll with an SSE stream and add a “Mom is online right now” badge on her family card. This was the single most requested feature in informal user testing.

Encryption-at-rest for OAuth tokens. A process.env-keyed AES round on insert, a decrypt on read.

Supertest integration suite for the API. Two days of work, catches a regression class the unit suite cannot.

DST edge-case fix for the wall-clock anchor. Detailed in Section 7.1.

Redis-backed rate limiter for multi-process deployment.

Mobile feature parity. Notes, Google Calendar OAuth, 2FA enrolment ported from web to the Expo app.

Two-way Google Calendar sync. Write schedule_blocks back as Google Calendar events (a different OAuth scope and a more interesting threat model than the v1 read-only import).

Items explicitly not planned for v1.1: Apple CalDAV (Section 7.6), in-app voice or video (the project is not trying to be Zoom), and end-to-end encryption (the project is not trying to be Signal).

8.3 Closing

The problem is not large. The system is not large. The contribution this report makes is not a new algorithm — set intersection, log- weighted scoring, and Intl.DateTimeFormat projection are all textbook. The contribution is the *framing*: that family communication across timezones is a problem worth treating as a recommendation problem rather than a calendar problem, that the right output is a ranked list of windows rather than a chart, and that the right level of privacy is per-contact rather than all-or-nothing.

Building TimeBridge convinced me that the framing matters more than the algorithms. I hope this report convinces the reader of the same.

Appendix A — Full REST API

23 endpoints. All authenticated routes (everything except /register, /login, /forgot-password, /reset-password, and the OAuth callback) require Authorization: Bearer <jwt>.

Method

Path

Description

POST

/register

Create account. Body: {name, email, password, timezone, city, country}.

POST

/login

Returns {token, user} or {twofa_required: true}. Rate-limited (10 / 15 min / IP).

POST

/forgot-password

Sends (or logs) a reset link. Rate-limited (5 / hour / IP).

POST

/reset-password

Body: {token, password}.

GET

/me

Current user profile (incl. city, country).

PUT

/me

Update profile (name, timezone, city, country).

DELETE

/me

Delete account + all owned data.

POST

/change-password

Verify current password (bcrypt), set new.

GET

/me/export

Download full user data as one JSON file.

GET

/privacy

Privacy settings (global + per-contact JSONB).

PUT

/privacy

Upsert one privacy row. Body: {contact_id, settings}.

POST

/2fa/setup

Issues a fresh TOTP secret + otpauth:// URL.

POST

/2fa/verify

Verifies the user’s first 6-digit code; sets twofa_enabled = true.

DELETE

/2fa

Disables 2FA. Body: {current_password}.

POST

/connections/invite

Generate a fresh 8-char invite code.

POST

/connections/join

Accept an invite. Body: {invite_code}. Now handles already-connected case.

GET

/connections

List your connections (pending + accepted, with each other party’s profile + city).

DELETE

/connections/:id

Revoke a pending invite, or unlink an accepted one.

GET

/connections/:otherId/availability

Other user’s availability slots (if connected).

POST

/availability

Replace your slots. Body: {slots: [iso, …]}.

GET

/availability

Your own slots.

GET

/availability/overlap

Slots where you and at least one connection are free.

GET

/availability/overlap?with=ID

Slots where you and *that specific* connection are free.

GET

/schedule

Recurring weekly blocks (incl. tzid).

POST

/schedule

Add a block (tzid optional, defaults to user's timezone).

DELETE

/schedule/:id

Remove a block.

GET

/integrations/google/status

Whether Google OAuth is configured + whether this user is connected.

GET

/integrations/google/connect

Returns {url} to send the user to Google's consent screen.

GET

/integrations/google/callback

OAuth redirect target. Exchanges code for tokens.

POST

/integrations/google/import

Pulls the next 30 days of timed events into schedule_blocks.

DELETE

/integrations/google

Removes stored Google tokens.

POST

/notes

Send a note. Body: {to_user_id, body}. Recipient must be a connection.

GET

/notes

Inbox + sent. Marks unread inbox notes as read in the same call.

GET

/notes/unread-count

Cheap count for the sidebar badge.

DELETE

/notes/:id

Delete a note. Sender or recipient.

Appendix B — Database Schema

Six core tables plus one auxiliary (password_resets). Schema is created idempotently in ensureSchema() on server startup; every ALTER TABLE uses ADD COLUMN IF NOT EXISTS so the server is safe to restart against an older database.

```
-- Users
CREATE TABLE users (
  id              SERIAL PRIMARY KEY,
  name            TEXT    NOT NULL,
  email           TEXT    UNIQUE NOT NULL,
  password_hash   TEXT    NOT NULL,
  timezone        TEXT    NOT NULL,
  city            TEXT,
  country         TEXT,
  twofa_secret    TEXT,
  twofa_enabled   BOOLEAN NOT NULL DEFAULT false,
  created_at      TIMESTAMP NOT NULL DEFAULT NOW()
);

-- Connections (pending + accepted)
CREATE TABLE connections (
  id                SERIAL PRIMARY KEY,
  user_id           INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  connected_user_id INTEGER          REFERENCES users(id) ON DELETE CASCADE,
  invite_code       TEXT    UNIQUE NOT NULL,
  status            TEXT    NOT NULL,            -- 'pending' | 'accepted'
  accepted_at       TIMESTAMP
);

-- Availability (one row per UTC hour)
CREATE TABLE availability (
  id          SERIAL PRIMARY KEY,
  user_id     INTEGER     NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  start_time  TIMESTAMPTZ NOT NULL
);

-- Recurring weekly schedule blocks
CREATE TABLE schedule_blocks (
  id          SERIAL PRIMARY KEY,
  user_id     INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  title       TEXT    NOT NULL,
  type        TEXT    NOT NULL DEFAULT 'other',
  color       TEXT,
  days        TEXT[]  NOT NULL,            -- e.g. ['Mon','Tue','Thu']
  start_time  TIME    NOT NULL,
  end_time    TIME    NOT NULL,
  tzid        TEXT,                        -- IANA timezone the block is
```

```sql
anchored to
  created_at  TIMESTAMP NOT NULL DEFAULT NOW()
);

-- Password resets (single-use, hashed)
CREATE TABLE password_resets (
  id          SERIAL PRIMARY KEY,
  user_id     INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  token_hash  TEXT    NOT NULL UNIQUE,
  expires_at  TIMESTAMP NOT NULL,
  used        BOOLEAN NOT NULL DEFAULT false,
  created_at  TIMESTAMP NOT NULL DEFAULT NOW()
);

-- Privacy settings (one per (user, contact))
CREATE TABLE privacy_settings (
  id          SERIAL PRIMARY KEY,
  user_id     INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  contact_id  INTEGER          REFERENCES users(id) ON DELETE CASCADE,
  settings    JSONB   NOT NULL,
  updated_at  TIMESTAMP NOT NULL DEFAULT NOW(),
  UNIQUE (user_id, contact_id)
);

-- Notes
CREATE TABLE notes (
  id            SERIAL PRIMARY KEY,
  from_user_id  INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  to_user_id    INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  body          TEXT    NOT NULL,
  created_at    TIMESTAMP NOT NULL DEFAULT NOW(),
  read_at       TIMESTAMP
);
CREATE INDEX idx_notes_to_user   ON notes (to_user_id,   created_at DESC);
CREATE INDEX idx_notes_from_user ON notes (from_user_id, created_at DESC);

-- OAuth integrations
CREATE TABLE integrations (
  id            SERIAL PRIMARY KEY,
  user_id       INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  provider      TEXT    NOT NULL,
  access_token  TEXT    NOT NULL,
  refresh_token TEXT,
  expires_at    TIMESTAMP,
  scope         TEXT,
  created_at    TIMESTAMP NOT NULL DEFAULT NOW(),
  UNIQUE (user_id, provider)
);
```

Appendix C — Architecture Diagram

A full-page reproduction of the three-tier architecture diagram is in docs/architecture.svg. The diagram shows:

Tier 1 (Client): React SPA (Vite), .ics import (in-browser), Browser Geolocation + Nominatim (reverse-geocode), Open-Meteo (weather), Expo mobile (subset).

Tier 2 (Backend): REST API (Express), Auth (JWT + bcrypt + TOTP), Overlap algorithm (UTC ISO + set intersection + coalesce + rank), SMTP (optional).

Tier 3 (Data): PostgreSQL + node-postgres connection pool, parameterised SQL.

Arrows: HTTPS + JWT (Client → Backend), JSON response (Backend → Client), SQL via pg pool (Backend → Postgres), rows (Postgres → Backend), async email send (Backend → SMTP), OAuth (Backend ↔ Google), HTTP fetch with CORS (Client → Open-Meteo + Nominatim).

Legend distinguishes internal components (solid border, blue), database / persistence (solid border, green), and external / third-party services (dashed border, amber).

Bibliography

Belshe, M., Peon, R., & Thomson, M. (2015). *Hypertext Transfer Protocol Version 2 (HTTP/2)*. RFC 7540, IETF. https://datatracker.ietf.org/doc/html/rfc7540X

Desruisseaux, B. (Ed.) (2009). *Internet Calendaring and Scheduling Core Object Specification (iCalendar)*. RFC 5545, IETF. https://datatracker.ietf.org/doc/html/rfc5545X

M'Raihi, D., Bellare, M., Hoornaert, F., Naccache, D., & Ranen, O. (2005). *HOTP: An HMAC-Based One-Time Password Algorithm*. RFC 4226, IETF. https://datatracker.ietf.org/doc/html/rfc4226X

M'Raihi, D., Machani, S., Pei, M., & Rydell, J. (2011). *TOTP: Time-Based One-Time Password Algorithm*. RFC 6238, IETF. https://datatracker.ietf.org/doc/html/rfc6238X

Hardt, D. (Ed.) (2012). *The OAuth 2.0 Authorization Framework*. RFC 6749, IETF. https://datatracker.ietf.org/doc/html/rfc6749X

Provos, N., & Mazières, D. (1999). *A Future-Adaptable Password Scheme*. USENIX Annual Technical Conference. https://www.usenix.org/legacy/event/usenix99/provos/provos.pdfX

Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON Web Token (JWT)*. RFC 7519, IETF. https://datatracker.ietf.org/doc/html/rfc7519X

The PostgreSQL Global Development Group. (2024). *PostgreSQL 14 Documentation*. https://www.postgresql.org/docs/14/X

Meta Platforms. (2024). *React 18 Documentation*. https://react.dev/X

OpenJS Foundation. (2024). *Express 5 Documentation*. https://expressjs.com/X

Open-Meteo. (2024). *Free Weather API*. https://open-meteo.com/X

OpenStreetMap Foundation. (2024). *Nominatim Geocoder*. https://nominatim.org/X

Vite team. (2024). *Vite 5 Documentation*. https://vitejs.dev/X

Lucide contributors. (2024). *Lucide Icon Library*. https://lucide.dev/X

WorkOS / Radix UI team. (2024). *Radix UI Primitives*. https://www.radix-ui.com/primitivesX